Talent-DZ

Algerian Recruitment Platform

Real-time chat. Instant decisions.

TECH STACK

Next.js 15 | Supabase | Vercel | TypeScript

Architect README • Mission 4

Cloud Architecture & Real-Time Realtime Engineering

Information Systems • ESTIN
2025/2026 Academic Year

Contents

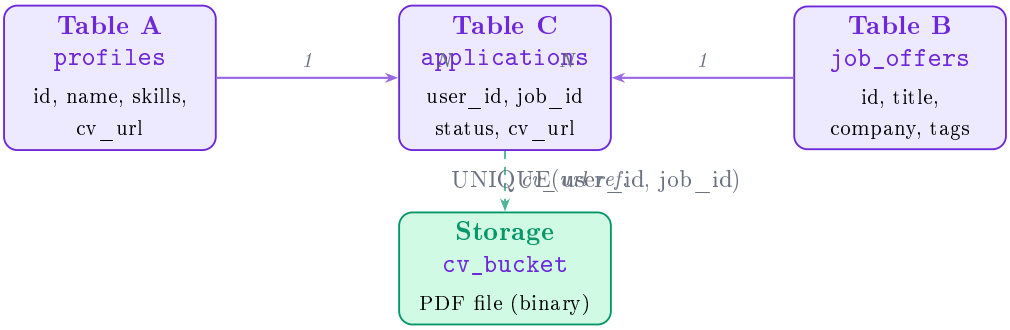
1. Theme Mapping

Talent-DZ is a job platform connecting Algerian candidates with employers. Users browse verified listings, upload their CV once, and apply with a single click — pipeline status tracked in real time.

Database Tables — A, B, C and Storage

Entity	Table	Purpose	Type
Table A	`profiles`	Linked to Supabase Auth via `auth.users`. Stores name, bio, location, skills, and CV reference. Auto-created by a trigger on every sign-up.	Users
Table B	`job_offers`	Browsable job listings. Holds title, company, description, salary, location, contract type, and skill tags. Publicly readable without login.	Resources
Table C	`applications`	Join table linking a candidate (A) to a listing (B). Tracks pipeline status and stores the CV URL. A unique constraint prevents duplicate applications.	Interactions
Storage	`cv_bucket`	PDF CV — the unstructured binary file. Uploaded once, reused across applications. Isolated by RLS so no user can access another candidate's file.	Files

Database Relationship Diagram



User Flow

2. Architecture Analysis

2.1. Q1 — CAPEX vs. OPEX

Physical server requires large upfront CAPEX: hardware, cooling, licenses. **Vercel + Supabase** are OPEX: pay only for usage, month to month. For Talent-DZ, the free tier covers the full demo; no upfront cost even if 500 candidates register day one.

2.2. Q2 — Scalability: Vercel Edge

Vercel routes requests to nearest edge node in milliseconds. Backend functions spin up per request, then vanish — no idle server. Automatic horizontal scaling. Supabase same: PostgreSQL scales on demand; Realtime broadcasts handled by managed infrastructure.

2.3. Q3 — Structured vs. Unstructured

Structured: Profiles, job offers, applications, messages all in PostgreSQL. Queryable via SQL. **Unstructured:** CV files in Storage (binary PDF). Invisible to SQL. Linked via `cv_url`. Each tool does one job well.

3. Database Schema — Mission 1

Table A — `profiles` (Users / Candidates)

Column	Type	Constraint	Description
`id`	UUID	PK, FK `auth.users`	Auto-linked to Supabase Auth user
`first_name`	TEXT	—	Candidate first name
`last_name`	TEXT	—	Candidate last name
`role`	TEXT	DEFAULT `candidate`	User role
`bio`	TEXT	—	Short biography
`location`	TEXT	—	City / wilaya
`skills`	TEXT[]	—	Array of skill tags
`cv_url`	TEXT	—	Signed URL to Storage CV
`created_at`	TIMESTAMP	—	Account creation date

Table B — `job_offers` (Listings / Resources)

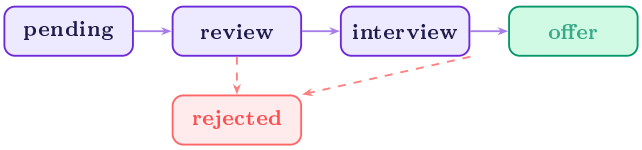
Column	Type	Constraint	Description
`id`	UUID	PK	Auto-generated identifier
`title`	TEXT	NOT NULL	Job title
`company`	TEXT	NOT NULL	Employer name
`description`	TEXT	—	Full job description
`salary`	TEXT	—	Salary range (e.g. 150k–220k DA)
`location`	TEXT	—	City / wilaya
`type`	TEXT	—	Full-time / Remote / Hybrid
`tags`	TEXT[]	—	Required skills array
`created_at`	TIMESTAMP	—	Listing date

Table C — `applications` (Join Table / Interactions)

Column	Type	Constraint	Description
`id`	UUID	PK	Auto-generated identifier
`user_id`	UUID	FK `profiles`	Links to candidate (Table A)
`job_id`	UUID	FK `job_offers`	Links to listing (Table B)
`status`	TEXT	DEFAULT `pending`	Pipeline stage
`cv_url`	TEXT	—	URL of submitted CV
`created_at`	TIMESTAMP	—	Application date

UNIQUE(user_id, job_id) — one application per listing per candidate

Application status pipeline:



Storage — `cv_bucket`

Property	Value
Bucket name	`cv_bucket`
Public access	No (private)
File type	PDF (CV documents)
Access control	Row Level Security — each user reads/writes only their own files
Link to DB	`cv_url` column in `profiles` and `applications`

Row Level Security Policies

Table	Policy	Operation	Rule
profiles	See own profile	SELECT	auth.uid() = id
profiles	Edit own profile	UPDATE	auth.uid() = id
job_offers	Public listings	SELECT	true (no auth needed)
applications	View own	SELECT	auth.uid() = user_id
applications	Create own	INSERT	auth.uid() = user_id
applications	Update own	UPDATE	auth.uid() = user_id
cv_bucket	Upload / read	ALL	auth.uid() = owner

SQL Listings

Listing 1 — Table definitions (A, B, C)

```sql
-- Table A : Users / Candidates
CREATE TABLE public.profiles (
  id          UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
  first_name TEXT,  last_name TEXT,
  role        TEXT DEFAULT 'candidate',
  bio TEXT,  location TEXT,  phone TEXT,
  skills      TEXT[],
  cv_name     TEXT,  cv_url TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', now())
);

-- Table B : Job Offers (Resources)
CREATE TABLE public.job_offers (
  id           UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  title        TEXT NOT NULL,  company TEXT NOT NULL,
  description TEXT,  salary TEXT,  location TEXT,  type TEXT,
  tags         TEXT[],
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', now())
);

-- Table C : Applications (A x B join)
CREATE TABLE public.applications (
  id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id     UUID REFERENCES auth.users        ON DELETE CASCADE NOT NULL,
  job_id      UUID REFERENCES public.job_offers ON DELETE CASCADE NOT NULL,
  status      TEXT DEFAULT 'pending', -- pending|review|interview|offer|
    rejected
  cv_url      TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', now()),
  UNIQUE (user_id, job_id)
);
```

Listing 2 — Auto-profile trigger on sign-up

```sql
CREATE OR REPLACE FUNCTION public.handle_new_user()
```

4. Chat Backend Realtime Architecture — Mission 4

4.1. 1. Project Vision

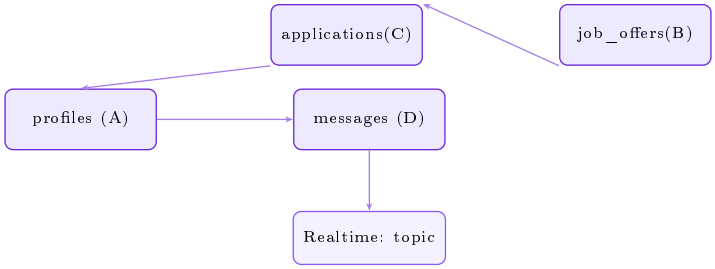
Talent-DZ enables real-time peer-to-peer messaging between candidates and recruiters. Messages are stored in public.messages and broadcast live via Supabase Realtime — a WebSocket event

stream. Both participants receive updates instantly without page refresh using deterministic topic routing.

4.2. 2. Data Model

Table	Role	Chat Context
`profiles` (A)	User identity	Message sender/receiver
`job_offers` (B)	Context	Recruitment topic
`applications` (C)	Interaction trigger	Chat begins after apply
`messages` (D)	Chat payload	sender_id, receiver_id, content, created_at

Data Flow (A→D, B→C→messages):



4.3. 3. End-to-End Realtime Flow

INSERT → Trigger → Broadcast → Topic → Merge:

1. Frontend POSTs message to `/api/chat/send`
2. Backend INSERTs row into `public.messages`
3. AFTER trigger fires, calls `messages_broadcast_changes()`
4. Trigger computes: `conversation:least(sender,receiver):greatest(sender,receiver)`
5. Trigger calls `realtime.broadcast_changes()` with operation + payload
6. Frontend subscribed to topic receives event
7. Frontend extracts `operation` and `record`, merges by `id`
8. UI updates in `created_at` chronological order

4.4. 4. Security: RLS + Realtime Authorization

RLS on `public.messages`:

Policy	Op	Condition
View own	SELECT	`auth.uid() = sender_id OR receiver_id`
Send	INSERT	`auth.uid() = sender_id`

Realtime Delivery Policy: `realtime.messages` → `authenticated_can_receive_broadcasts` (SELECT: USING (true)). This permits all authenticated users to receive broadcasts; actual row filtering is enforced by source RLS.

4.5. 5. Topic Strategy

Deterministic topic ensures both participants subscribe to identical channel:

```
conversation:min(sender_id, receiver_id):max(sender_id, receiver_id)
```

Topic is the Realtime routing key, not an authorization boundary. RLS + delivery policy enforce access.

4.6. 6. Payload Contract

Field	Value
`payload.operation`	"INSERT", "UPDATE", "DELETE"
`payload.record`	New/updated record (INSERT/UPDATE)
`payload.old_record`	Old record (UPDATE only)

Uncertainty: Envelope structure varies. Frontend defensively tries: `payload.payload.operation`, `payload.eventType`, `payload.event`. **Verify by logging raw event:** `console.log(JSON.stringify(incoming))`

4.7. 7. Failure Modes

Mode	Symptom	Debug / Root
Trigger not firing	No event appears	Check Postgres logs; verify AFTER INSERT trigger
Topic mismatch	No events received	Log both topic strings; verify least/greatest
Payload envelope mismatch	Record is null	Log raw incoming; inspect paths in `getRecordFromPayload()`
RLS blocks INSERT	403 error	Verify sender_id = auth.uid(); check policy
Duplicate merge	Msg appears twice	Verify map-based dedup by `id`
Ordering mismatch	Out of sequence	Check sort by `created_at`; verify frontend sort

4.8. 8. Verification Checklist

1. **DB:** `public.messages` table exists; trigger fires on INSERT
2. **Trigger:** Verify `broadcast_changes()` call in function
3. **Topic:** Frontend + backend logs show identical topic string
4. **RLS:** Can't SELECT messages sent by others (blocked)
5. **Realtime:** Two browser windows subscribed to same topic; msg from one appears in other
6. **CRITICAL — Payload Parsing:** Log `console.log(JSON.stringify(incoming))` on first event; inspect structure vs extraction paths
7. **Dedup:** Send same message ID twice; UI shows only one copy
8. **Order:** Send 3 messages; verify UI sorts ascending by `created_at`

Key Issue: Payload envelope mismatch. Always log raw incoming event first. Exact field paths vary by Supabase version.

Listing 3 — Auto-profile trigger on sign-up

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
  INSERT INTO public.profiles (id) VALUES (new.id);
  RETURN new;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE OR REPLACE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE PROCEDURE public.handle_new_user();
```

Listing 3 — Row Level Security

```
ALTER TABLE public.profiles     ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.job_offers   ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.applications ENABLE ROW LEVEL SECURITY;

```

```sql
CREATE POLICY "View own profile"   ON public.profiles     FOR SELECT USING (auth.uid() = id);
CREATE POLICY "Edit own profile"   ON public.profiles     FOR UPDATE USING (auth.uid() = id);
CREATE POLICY "Public job listings" ON public.job_offers  FOR SELECT USING (true);
CREATE POLICY "View own applications" ON public.applications FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Create application"    ON public.applications FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Update own application" ON public.applications FOR UPDATE USING (auth.uid() = user_id);
```

Listing 4 — Storage bucket & RLS

```sql
INSERT INTO storage.buckets (id, name, public)
VALUES ('cv_bucket', 'cv_bucket', false)
ON CONFLICT (id) DO NOTHING;

CREATE POLICY "Upload own CV" ON storage.objects FOR INSERT
  WITH CHECK (bucket_id = 'cv_bucket' AND auth.uid() = owner);
CREATE POLICY "Read own CV"   ON storage.objects FOR SELECT
  USING  (bucket_id = 'cv_bucket' AND auth.uid() = owner);
```

5. Tech Stack & Project Links

Layer	Tool	Role
Frontend	Next.js 15 (App Router)	React UI, routing, Server Components
Styling	Tailwind CSS + CSS vars	Design system with violet palette
Auth	Supabase Auth	JWT sessions, auto profile trigger
Database	Supabase (PostgreSQL)	Tables A/B/C, FK constraints, RLS
Storage	Supabase Storage	PDF CVs, per-user isolation
CI/CD	Vercel + GitHub	Auto-deploy on every `git push`

Production https://si-project-recretement-one.vercel.app
Repository https://github.com/khadrabesmala/si-project-recretement-
Test login besamalasaada@gmail.com / 12345678